

CSE 312 — İşletim Sistemleri

Memory Management

Konu: Ana Bellekten Sanal Bellek Yönetimine

Lecture 06 — Spring 2026

Kapsamlı Türkçe Konu Anlatımı

İçindekiler

- 1.** Bellek Yönetimine Giriş
- 2.** Bellek Bir Kaynak Olarak
- 3.** Statik Overlay Yapısı
- 4.** Çok Programlama ve Bellek Sorunları
 - 4.1 Yer Değiştirme (Relocation)
 - 4.2 Bellek Koruma (Memory Protection)
 - 4.3 Tahsis (Allocation)
- 5.** Yer Değiştirme Çözümleri
 - 5.1 Konumdan Bağımsız Kod (PIC)
 - 5.2 Yükleme Zamanı Yer Değiştirme
 - 5.3 Donanım Destekli — Base/Limit Register
- 6.** Swapping (Takas)
- 7.** Bellek Tahsis Algoritmaları
 - 7.1 First Fit
 - 7.2 Best Fit
 - 7.3 Worst Fit
- 8.** Sanal Bellek (Virtual Memory)
 - 8.1 Kavramsal Temel
 - 8.2 Sanal Bellek Yönetimi & Sayfa Tablosu
- 9.** Quiz Soruları ve Çözümleri

1. Bellek Yönetimine Giriş

Bellek yönetimi (memory management), işletim sistemlerinin en kritik görevlerinden biridir. Programların çalışabilmesi için ana bellekte (RAM) yer kaplaması gerekir; ancak bellek sınırlı bir kaynaktır. İşletim sistemi bu kaynağı verimli biçimde paylaşdırmak zorundadır.

Basit bir sistemde, işletim sistemi belleğin alt kısmını kaplar ve uygulama geri kalan alanı kullanır. Ancak çok programlı (multiprogramming) ortamlarda durum çok daha karmaşık hale gelir.

Adres	İçerik
64K ↑	Kullanılmayan Alan (Unused)
~32K	Uygulama 3
~20K	Uygulama 2
~10K	Uygulama 1
0	İşletim Sistemi (OS)

Şekil 1: Çok Programlı Sistemde Bellek Düzeni

► İşletim Sisteminin Bellek Yönetim Görevleri

Hangi bellek alanlarının kullanımda olduğunu takip etme

Bellek talep eden süreçlere alan tahsis etme

İşi biten süreçlerin belleğini serbest bırakma

Belleğin dolması durumunda takas (swapping) mekanizmasını devreye alma

2. Bellek Bir Kaynak Olarak

Bellek, bilgisayarın ilk günlerinden bu yana kıt bir kaynak olagelmıştır. Daha doğrusu, belleğin maliyeti genel olarak yüksek seyretmiş; mevcut bellek kapasitesi hiçbir zaman tüm uygulamaların ihtiyacını karşılamaya yetmemiştir. Bu yüzden yazılım ve donanım tasarımcıları sürekli yeni çözümler üretmek zorunda kalmıştır.

Tarihin her döneminde programcılar bellek sınırlamalarıyla başa çıkmak için farklı teknikler geliştirmiştir. Bu teknikler zaman içinde manuel süreçlerden donanım destekli otomatik çözümlere (sanal bellek) evrilmiştir.

► Tarihsel Perspektif

1950'ler-60'lar: Programcı belleği elle yönetirdi (overlay yapıları).

1960'lar-70'ler: Çok programlama → OS bellek yönetimi zorunlu hale geldi.

1970'ler-günümüz: Sanal bellek → programcı bellek sınırlarını büyük ölçüde görmez.

3. Statik Overlay Yapısı

İlk dönem sistemlerde, uygulama programcıları bellek tahsisini tamamen kendileri yapmak zorundaydı. Bellek çok küçük olduğundan, bir programın farklı alt rutinleri (subroutine) aynı bellek bölgesini farklı zamanlarda paylaşıyordu. Bu tekniğe **statik overlay** denir.

İşletim sistemi bu işlemi desteklemek için kütüphane rutinleri sağlardı, ancak hangi rutinin ne zaman yükleneceğine ve güvenliğine karar vermek tamamen programcıya aitti. Bu yaklaşım hata yapmaya son derece açıktı.

► Statik Overlay — Örnek Senaryo

Program toplam 64 KB bellek gerektiriyor; RAM yalnızca 32 KB.

Çözüm: Modül A ve Modül B aynı 16 KB alanı paylaşır.

A çalışırken B diskte bekler, B çalışırken A diskte bekler.

Programcı hangi modüllerin aynı anda gerekli olmadığını bilmek zorundadır.

4. Çok Programlama ve Bellek Sorunları

Bellek kapasitesinin artmasıyla birlikte, birden fazla programın eş zamanlı olarak bellekte tutulması ve CPU'yu paylaşması mümkün hale geldi. Bu yaklaşıma **multiprogramming** (çok programlama) denir. Ancak bu durum işletim sistemi için yeni ve zorlu sorunlar ortaya çıkardı.

Sorun	Açıklama	Etki
Yer Değiştirme (Relocation)	Program başlangıç adresi her çalışmada farklı olabilir	Bellek adresleri derleme zamanında sabitlenememez
Bellek Koruma (Protection)	Bir process başka bir process'in belleğine erişmemeli	Doğru izolasyon olmadan sistem çöker
Tahsis (Allocation)	Hangi boş bellek bloğu hangi sürece verilmeli?	Parçalanma (fragmentation) ve israf

4.1 Yer Değiştirme (Relocation)

Erken sistemlerde tüm uygulamalar aynı başlangıç adresinden (örneğin 16K) yükleniyordu. Çok programlamada ise bir uygulama her defasında farklı bir adresten başlayabilir. Bu durum şu soruyu doğurur: **Programa ait bellek referansları nasıl doğru adresleri gösterecek?**

Üç temel çözüm yaklaşımı geliştirilmiştir:

- **Konumdan Bağımsız Kod (Position-Independent Code, PIC)**
- **Yükleme Zamanı Yer Değiştirme (Load-Time Relocation)**
- **Donanım Destekli Yer Değiştirme (Hardware-Assisted Relocation)**

4.2 Bellek Koruma (Memory Protection)

Çok programlamada birden fazla süreç aynı anda bellekte bulunur. Bir sürecin başka bir sürecin (ya da işletim sisteminin) belleğine erişmesi hem güvenlik hem de kararlılık açısından tehlikelidir. Bu nedenle donanım düzeyinde koruma mekanizmaları geliştirilmiştir.

► IBM System/360 Çözümü

Her 4 KB'lık bellek bloğuna 4 bitlik bir 'depolama anahtarı' (storage key) atandı.

Program Durum Sözcüğü'ndeki (PSW) anahtarla eşleşmeyen erişimler reddedildi.

Bu, donanım tabanlı bellek korumasının ilk örneklerinden biridir.

4.3 Tahsis (Allocation)

İşletim sistemi, bellekte hangi alanların boş olduğunu takip eder ve bir uygulama bellek istediğinde uygun bloğu seçmek zorundadır. Bu seçim için genellikle bağlı liste (linked list) veri yapısı kullanılır. Bölüm 7'de çeşitli tahsis algoritmaları ayrıntılı ele alınacaktır.

5. Yer Değiştirme Çözümleri

5.1 Konumdan Bağımsız Kod (PIC)

Bazı kodlar doğası gereği konumdan bağımsızdır çünkü mutlak bellek adreslerine değil, bir yazmaç (register) veya program sayacına (program counter, PC) göre **görelî (relative)** referanslar kullanırlar. Bu tür kod, nereye yüklenirse yüklensin herhangi bir değişiklik gerektirmez.

Modern Unix/Linux sistemlerinde paylaşılan kütüphaneler (shared libraries) PIC kullanır. GCC'de C(-fpic) veya C(-fPIC) seçeneğiyle etkinleştirilir.

► PIC — Avantaj ve Dezavantaj

- ✓ Yükleme sonrası hiçbir düzeltme gerekmez.
- ✓ Paylaşılan kütüphaneler için idealdir.
- ✗ Bazı donanımlarda yazmaç kullanımı nedeniyle küçük performans kaybı.
- ✗ Yükleme sonrası program başka bir adrese taşınırsa yine de geçersiz olur.

5.2 Yükleme Zamanı Yer Değiştirme (Load-Time Relocation)

Program yüklendiğinde çalışacağı gerçek adres bilinir. Yükleyici (loader), derleyici/bağlayıcının varsaydığı adreslere gerçek ofseti ekler. Bu işlem, yeniden düzenlenmesi gereken sözcük sayısına bağlı olarak donanım mimarisine ve derleyiciye göre değişen bir maliyete sahiptir.

► Load-Time Relocation — Nasıl Çalışır?

- Derleyici, programın 0x0000'dan başladığını varsayarak derleme yapar.
- Bağlayıcı (linker), yer değiştirilmesi gereken referansları bir tabloda listeler.
- Yükleyici programı 0x4000'e yükleyecekse tablodaki tüm adreslere 0x4000 ekler.
- Yükleme maliyeti: Ne kadar çok mutlak referans varsa o kadar çok düzeltme gerekir.

5.3 Donanım Destekli Yer Değiştirme — Base/Limit Register

PIC bazen kullanışsızdır ve yükleme zamanı yer değiştirme pahalı olabilir. Ayrıca programları çalışma zamanında taşımak gerekebilir. Donanım tabanlı çözüm olarak **Taban/Sınır Yazmaçları (Base/Limit Registers)** geliştirilmiştir.

İşleyiş: İşletim sistemi **Taban (Base)** ve **Sınır (Limit)** yazmaçlarını ayarlar. Donanım her bellek referansına taban değerini ekler ve sonucun limit değerini aşıp aşmadığını kontrol eder. Sınır aşırsa donanım bir **trap** (kesme) üretir.

► Base/Limit Register — Sorunlar

- Program kopyalama pahalıdır (tüm bellek içeriği taşınmalı).
- Her bellek referansında ekstra bir toplama işlemi gerekir → yavaşlık.
- Daha fazla esneklik gerektirir: Birden fazla segment, farklı koruma seviyeleri...

6. Swapping (Takas)

Swapping, bellekte yer açmak amacıyla bir sürecin tamamını diske yazma ve ardından çalışmaya hazır olduğunda geri okuma işlemidir. Bu sayede fiziksel bellekten daha büyük bir toplam bellek alanına sahipmiş gibi davranılabilir.

Adım	İşlem	Etki
1	Bloke olan (blocked) süreç seçilir	CPU'yu kullanamayan süreç diske gönderilir
2	Süreç tümüyle diske yazılır (swap-out)	Bellek alanı serbest kalır
3	Başka bir süreç bu alana yüklenir	Farklı bir adrese yüklenebilir
4	İlk süreç hazır olunca diskten okunur (swap-in)	Bellek sıkıştırması (compaction) mümkün

► Swapping'in Faydası ve Bedeli

- ✓ Belleği sıkıştırmak (compaction) mümkün hale gelir.
- ✓ Bellekte yer olmasa da daha fazla süreç çalıştırılabilir görünür.
- ✗ Disk G/Ç (I/O) işlemleri bellek bantgenişliğini (bandwidth) tüketir.
- ✗ Tüm sürecin yazılıp okunması çok yavaştır — bugün artık sayfa bazlı yapılır.

7. Bellek Tahsis Algoritmaları

İşletim sistemi boş bellek bloklarını bir bağlı listede tutar. Bir uygulama N byte bellek istediğinde, OS bu listeden uygun bloğu seçmelidir. Yanlış seçim, bellekte parçalanmaya (fragmentation) yol açar.

7.1 First Fit (İlk Uygun)

Listeyi baştan tara; yeterince büyük ilk bloğu bul, ihtiyaç kadar al, gerisini serbest listeye geri koy. **Serbest bloklar adrese göre sıralı listede tutulur** (bitişik blokları birleştirmek — coalesce — için).

► First Fit — Değerlendirme

- ✓ Hızlıdır; tam eşleşme aramak gerekmez.
- ✓ Pratikte Best Fit kadar iyi sonuç verir.
- ✗ Zamanla bellekte küçük, işe yaramaz boşluklar birikir (dış parçalanma).
- ✗ Belleğin 1/3'ü kullanılamaz hale gelebilir (simülasyon sonucu).

7.2 Best Fit (En İyi Uygun)

Boyuta göre sıralı listede isteğe en yakın boyuttaki bloğu bul. Sezgisel olarak First Fit'ten daha verimli görünse de **pratikte değildir**: çok sayıda küçük, işe yaramaz artık blok bırakır.

► Best Fit — Değerlendirme

- ✓ Tam eşleşme durumunda hiç artık kalmaz.
- ✗ Tüm listenin taranması gerekir → First Fit'ten yavaş.
- ✗ Bıraktığı küçük artıklar (fragmentation) daha kötüdür.
- ✗ Bitişik blokları birleştirmek (coalesce) daha zordur (liste boyuta göre sıralı).

7.3 Worst Fit (En Kötü Uygun)

En büyük boş bloğu seç ve parçala. Fikir şudur: Büyük bir bloğu parçalamak, kullanışsız küçük artıklar yerine yine kullanışlı büyük bir artık bırakır. **Ancak simülasyonlar, First Fit ve Best Fit'in Worst Fit'ten daha iyi sonuçlar verdiğini göstermektedir.**

► Algoritma Karşılaştırması (Simülasyon Sonuçları)

First Fit $\approx$ Best Fit $>$ Worst Fit (performans açısından)

First Fit: Belleğin $\sim 1/3$ 'ü parçalanma nedeniyle kullanılamaz hale gelebilir.

Best Fit: Küçük, birleştirilemeyen artıklar birikir.

Worst Fit: Büyük bloklar yok edilir → en büyük süreçler çalıştırılmaz.

Algoritma	Veri Yapısı	Hız	Parçalanma	Pratik Kullanım
First Fit	Adrese göre sıralı bağlı liste	Hızlı ($O(n)$)	Orta	Yaygın
Best Fit	Boyuta göre sıralı bağlı liste	Yavaş ($O(n)$)	Kötü	Nadiren
Worst Fit	Boyuta göre sıralı (tersten)	Yavaş ($O(n)$)	En Kötü	Hemen hiç

► Günümüzde Bellek Tahsisi

Klasik OS bellek tahsisi Sanal Bellek nedeniyle çok önemli değil.

Ancak uygulama düzeyinde (malloc/free, new/delete) aynı algoritmalar kullanılıyor.

Uygulamalar ihtiyaç duydukça OS'tan sayfa (page) isteyebilir.

8. Sanal Bellek (Virtual Memory)

8.1 Kavramsal Temel

Daha önceki tüm yaklaşımların ortak sorunları vardı: programcı zamanı israfı, blok tahsis algoritmalarının verimsizliği, sıkıştırma (compaction) maliyeti, taban yazmaçlarının masrafı ve özel koruma mekanizmaları ihtiyacı. **Sanal bellek tüm bu sorunları tek bir soyutlama ile çözer.**

► Sanal Belleğin Çözdüğü Sorunlar

Programcı bellek tahsimiyle uğraşmak zorunda kalmaz.

Blok tahsis algoritmalarının verimsizliği ortadan kalkar.

Sıkıştırma (compaction) gerekmez.

Taban yazmaçlarının masrafı yoktur.

Özel bellek koruma mekanizmaları büyük ölçüde gereksizleşir.

Temel fikir: Programın gördüğü **sanal adres (virtual address)** ile bellekteki gerçek **fiziksel adres (physical address)** birbirinden ayrılır. Bu eşleme **sayfalar (pages)** aracılığıyla yapılır; tipik sayfa boyutu 4 KB'tır.

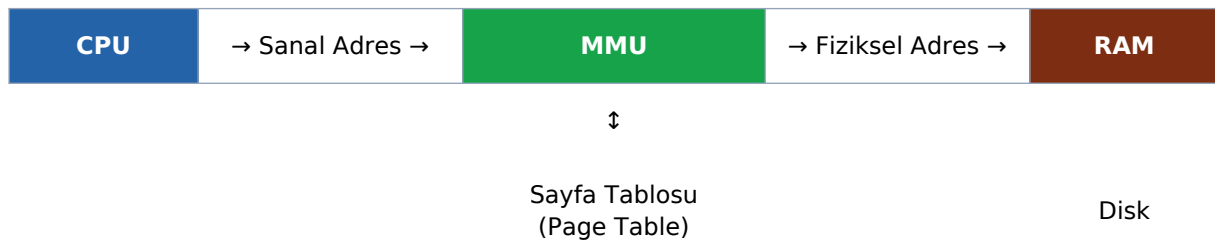
Bu eşleme sayesinde:

- Sanal adresler fiziksel olarak bitişik olmak zorunda değildir.
- Her uygulama aynı sanal adresten (örn. 0x0000) başlayabilir.
- Fiziksel belleğin gösterilmeyen kısımlarına erişilemez → doğal koruma.
- İşletim sistemi sanal adres alanında görünür olmayabilir.

Sanal bellek dönüşümü şu şekilde formüle edilir:

Bu dönüşüm, taban yazmacı yöntemine göre daha hızlıdır çünkü yalnızca bir tablo indeksi (dizi erişimi) gerektirir; toplama işlemi gerekmez.

Bellek Yönetim Birimi (MMU — Memory Management Unit): Sanal → Fiziksel adres dönüşümünü gerçekleştiren donanım birimidir. Tarihsel olarak ayrı bir yonga (chip) olarak tasarlanmış olsa da, günümüzde genellikle CPU çipinin içine entegre edilmiştir.



Şekil 2: MMU'nun Sistem İçindeki Konumu

8.2 Sanal Bellek Yönetimi ve Sayfa Tablosu

Sanal bellek yönetiminin kilit veri yapısı **sayfa tablosudur (page table)**. Her sürecin kendi sayfa tablosu vardır. Tablo, sanal sayfa numarasını fiziksel çerçeve numarasına (frame number) eşler.

Önemli özellikler:

- Sayfa tablosu, sürecin sanal bellek görüntüsündeki **her sayfa için bir giriş** içerir.
- **Tüm sayfalar diskte temsil edilir** (swap alanında veya çalıştırılabilir dosyada).
- Sayfaların yalnızca **bir kısmı fiziksel belleğe yüklenir** — gerisine gerektiğinde erişilir.

Aşağıdaki tabloda tipik bir sayfa tablosunun nasıl görüldüğü özetlenmiştir. Geçerli/geçersiz bit (valid/invalid bit), sayfanın fiziksel bellekte (RAM) mevcut olup olmadığını gösterir:

Sayfa No (Virtual)	Çerçeve No (Frame)	Geçerlilik Biti	Durum
0 (A)	4	v (geçerli)	RAM'de — erişilebilir
1 (B)	—	i (geçersiz)	Diskte — page fault gerekli
2 (C)	6	v (geçerli)	RAM'de — erişilebilir
3 (D)	—	i (geçersiz)	Diskte — page fault gerekli
4 (E)	—	i (geçersiz)	Diskte — henüz yüklenmedi
5 (F)	9	v (geçerli)	RAM'de — erişilebilir
6 (G)	—	i (geçersiz)	Diskte
7 (H)	—	i (geçersiz)	Diskte

Tablo 2: Örnek Sayfa Tablosu — A, C, F sayfaları RAM'de; diğerleri diskte

Bir sürece ait sayfa tablosu RAM'de saklanır. MMU, her bellek referansında bu tabloyu indeksler. Tablonun büyüklüğü sorun oluşturabilir: Örneğin 1 GB RAM ve 4 KB sayfa boyutu varsa **256.000 olası sayfa** vardır — bu kadar kayıt için o sayıda MMU yazmacı bulundurmak imkânsızdır.

► Sayfa Tablosu'nun Temel Özellikleri

Bir sürecin sanal adres uzayındaki her sayfa için bir giriş (entry) vardır.

Geçerli bit (valid/present bit): sayfa fiziksel bellekte mi, diskte mi?

Sayfa tablosu RAM'de tutulur, ancak hız için TLB (Translation Lookaside Buffer) ön belleği kullanılır.

Modern sistemlerde çok seviyeli sayfa tabloları (multilevel page tables) kullanılır.

Sanal belleğin en önemli avantajlarından biri, **fiziksel bellekten daha büyük bir sanal adres uzayı** sunabilmesidir. Bir sürecin tüm sayfaları aynı anda RAM'de olmak zorunda değildir. **Referans yerselliği (locality of reference)** sayesinde çoğu durumda yalnızca küçük bir sayfa kümesi aktif olarak kullanılır.

9. Quiz Soruları ve Çözümleri

► Bu bölüm hakkında

Aşağıdaki sorular sınav formatındadır — çoktan seçmeli değil, açık uçlu.

Her soruyu önce kendiniz yanıtlamayı deneyin, ardından çözümle karşılaştırın.

Sorular, ders materyalinin tüm bölümlerini kapsamaktadır.

S1. Multiprogramming (çok programlama) ortamında yer değiştirme (relocation) sorunu neden ortaya çıkar ve hangi üç yaklaşımla çözülebilir?

Multiprogramming'de birden fazla program aynı anda bellekte bulunur. Her program farklı bir adrese yüklenebileceğinden, derleme zamanında sabit kodlanan bellek adresleri geçersiz hale gelir.

Üç çözüm yaklaşımı:

1. Konumdan Bağımsız Kod (PIC): Mutlak adres yerine yazmaç/PC'ye göreli adres kullanılır. Yükleme sonrası herhangi bir düzeltme gerekmez.
2. Yükleme Zamanı Yer Değiştirme: Yükleyci, programın yükleneceği gerçek adresi bilir ve tüm mutlak referanslara ofseti ekler.
3. Donanım Destekli (Base/Limit Register): Taban yazmacı değeri her bellek referansına otomatik eklenir, sınır yazmacı ise geçersiz erişimleri engeller.

S2. First Fit, Best Fit ve Worst Fit algoritmalarını karşılaştırınız. Simülasyon sonuçlarına göre hangi algoritma daha iyi performans gösterir ve neden?

First Fit: Listeyi baştan tarar, ilk yeterli bloğu alır. Hızlıdır ($O(n)$) ancak zamanla başlarda daha fazla parçalanma birikir.

Best Fit: En küçük yeterli bloğu arar. Tüm liste taranır → yavaş. Çok küçük artıklar bırakır, dolayısıyla parçalanma daha kötüdür.

Worst Fit: En büyük bloğu parçalar. Büyük blokların yok olmasına neden olur.

Simülasyonlar bu algoritmanın en kötü performansı gösterdiğini ortaya koymuştur.

Sonuç: First Fit $\approx$ Best Fit $>$ Worst Fit (First Fit genellikle tercih edilir; Best Fit kadar iyi ama daha hızlı). Buna rağmen her iki algoritmayla da belleğin yaklaşık 1/3'ü parçalanma nedeniyle kullanılamaz hale gelebilir.

S3. Sanal bellek ile fiziksel bellek arasındaki temel fark nedir? Sayfa tablosu (page table) bu sistemde ne işlev görür?

Sanal bellek, programın gördüğü adres uzayıdır; bu adresler derleyici tarafından üretilir ve gerçek (fiziksel) RAM adreslerinden bağımsızdır. Fiziksel bellek ise gerçek RAM'dir.

MMU (Bellek Yönetim Birimi) her sanal adresi bir fiziksel adrese dönüştürür. Bu dönüşüm 'sayfalar' (genellikle 4 KB) üzerinden yapılır.

Sayfa tablosu işlevi:

- Sanal sayfa numarası (VPN) → Fiziksel çerçeve numarası (PFN) eşlemesini tutar.
 - Her girişte geçerlilik biti (valid/present bit) vardır: 1 = sayfa RAM'de, 0 = sayfa diskte.
 - Sayfanın diskte olması durumunda 'page fault' üretilir; OS sayfayı diske/diskten aktarır.
 - Her sürecin kendi sayfa tablosu vardır → süreç izolasyonu sağlanır.
-

S4. Swapping (takas) işlemi nedir? Avantajları ve dezavantajları nelerdir? Günümüzde nasıl kullanılmaktadır?

Swapping: Bloke olan veya düşük öncelikli bir sürecin tüm bellek içeriğinin diske yazılması (swap-out) ve çalışmaya hazır olduğunda geri okunması (swap-in) işlemidir.

Avantajlar:

- Daha fazla süreç aktif tutulabilir (çok programlama derecesi artar).
- Bellek sıkıştırması (compaction) mümkün olur: boşluklar birleştirilebilir.

Dezavantajlar:

- Tüm sürecin diske yazılması/okunması son derece yavaştır.
- Disk I/O bellek bant genişliğini tüketir.
- Farklı bir adrese swap-in yapılacaksa yer değiştirme gerekebilir.

Günümüzde: Tam süreç swapping'i artık nadiren kullanılır. Bunun yerine sayfa bazlı takas (demand paging) tercih edilir: yalnızca ihtiyaç duyulmayan bireysel sayfalar diske taşınır.

S5. Konumdan Bağımsız Kod (PIC) nedir, nasıl çalışır ve hangi durumlarda tercih edilir?

PIC (Position-Independent Code), mutlak bellek adreslerine değil; program sayacına (PC) veya yazmaçlara göreli adreslere kullanan koddur.

Nasıl çalışır:

- 'LOAD R3, 16(R12)' gibi komutlar, R12 yazmacının değerine göre adresi hesaplar.
- 'B *-4' gibi dallar, mevcut PC değerinden göreli atlar.
- Program herhangi bir fiziksel adrese yüklenebilir, herhangi bir düzeltme gerekmez.

Tercih edildiği durumlar:

1. Paylaşılan kütüphaneler (shared libraries / .so dosyaları): Aynı kütüphane farklı süreçlerin adres uzayında farklı adreslerde bulunabilir.
 2. Güvenlik: ASLR (Address Space Layout Randomization) ile birlikte kullanılır.
 3. GCC'de -fpic veya -fPIC bayraklarıyla etkinleştirilir.
-